

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 2

Выполнил:

Берулава Леон Алхасович

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

По выбранному варианту необходимо будет реализовать RESTful API средствами express + typescript (используя ранее написанный boilerplate).

Ход работы

В рамках лабораторной работы был использован ранее разработанный boilerplate на Express + TypeScript + TypeORM для создания полнофункционального RESTful API кулинарного приложения. Boilerplate предоставил готовую структуру проекта с четким разделением на слои моделей, контроллеров, маршрутов и сервисов что значительно ускорило процесс разработки и обеспечил соблюдение архитектурных принципов.

Основой для реализации API стала заранее подготовленная структура каталогов включающая папки entities для моделей данных, controllers для бизнес-логики, routes для определения endpoints, services для работы с репозиториями и config для конфигурационных файлов. Такая организация позволила сосредоточиться на реализации бизнес-логики вместо настройки базовой инфраструктуры проекта.

Конфигурация TypeScript из boilerplate была адаптирована под требования кулинарного приложения с включением поддержки декораторов TypeORM и строгой типизации для предотвращения ошибок времени выполнения. Настройки компилятора включали experimentalDecorators и emitDecoratorMetadata для корректной работы с аннотациями Entity, Column и других декораторов ORM.

Middleware из boilerplate включал готовые решения для обработки CORS, безопасности через Helmet, логирования HTTP запросов через Morgan и парсинга JSON тел запросов. Дополнительно была интегрирована централизованная обработка ошибок через custom error handler middleware что обеспечило единообразие формата ошибок во всем API.

Базовый CRUD функционал из boilerplate был расширен специфичными для кулинарного домена возможностями включая поиск рецептов по ингредиентам, фильтрацию по времени приготовления, категориям и уровню сложности. Реализованы дополнительные endpoints для получения популярных рецептов, статистики пользователей и агрегированных данных по категориям блюд.

Система валидации данных была усилена через кастомные валидаторы для проверки формата email адресов, корректности числовых параметров пагинации и существования связанных сущностей. Валидация реализована как на уровне TypeORM декораторов так и в контроллерах для обработки сложных бизнес-правил специфичных для кулинарного приложения.

Аутентификация и авторизация были реализованы через хэширование паролей с использованием bcrypt и создание защищенных endpoints требующих проверки прав доступа. Система безопасности включает валидацию входных данных, защиту от SQL инъекций через параметризованные запросы TypeORM и ограничение размера тел запросов для предотвращения DoS атак.

Интеграция с внешними сервисами была реализована через HTTP клиенты для взаимодействия между микросервисами включая проверку существования пользователей при создании рецептов и синхронизацию данных между сервисами. Обработка сетевых ошибок включает retry логику и graceful degradation при недоступности зависимых сервисов.

Пользовательский API включает endpoints для регистрации, аутентификации, управления профилями и получения информации о пользователях с поддержкой поиска по различным критериям. Endpoints следуют REST конвенциям с использованием соответствующих HTTP методов GET для чтения, POST для создания, PUT для обновления и DELETE для удаления ресурсов.

Рецептурный API предоставляет расширенный функционал для создания, редактирования и поиска рецептов с поддержкой сложных фильтров по категориям, времени приготовления, количеству порций и ингредиентам. Реализована пагинация с настраиваемым размером страницы и сортировка по различным полям включая популярность, дату создания и время приготовления.

Каталожный API обеспечивает управление справочными данными включая категории блюд и уровни сложности приготовления. API поддерживает иерархическую структуру категорий и предоставляет endpoints для получения полного каталога данных используемых в других частях системы для классификации рецептов.

Система ответов API унифицирована через использование общего интерфейса ApiResponse включающего поля success, data, message и error для обеспечения консистентности обработки ответов на клиентской стороне. Все ошибки возвращаются с соответствующими HTTP статус кодами и детализированными сообщениями включая временные метки для упрощения отладки.

Вывод

Использование boilerplate позволило сосредоточиться на реализации бизнес-логики кулинарного приложения получив готовую архитектурную

основу с best practices современной веб-разработки и обеспечив высокое качество кода с соблюдением принципов SOLID и clean architecture.